

CardioEquation — Mentor Progress Report

Date: March 16, 2026 | **Run:** 4 (Phase 1 Complete)

Project Overview

We are building **CardioEquation**, a patient-specific ECG generation system using a Diffusion Transformer (DiT-ECG-B, 265M parameters). The model takes a patient's historical ECG as context and generates new, realistic, personalized ECG signals. The system uses:

- **Stage 0:** Contrastive pre-training to learn patient identity embeddings
- **Stage 1:** Diffusion training to generate new ECGs conditioned on that identity

Results Across All Runs

Metric	Run 1	Run 2	Run 3*	Run 4	Real ECG Target
FFD ↓	1032	42.0	—	21.7	0
MMD ↓	0.661	0.383	—	0.432	0
HR MAE ↓ (bpm)	17.2	9.3	—	31.4 [!]	0
ReID Top-1 ↑	11.8%	5.9%	—	11.8%	100%
ReID Top-5 ↑	35.3%	23.5%	—	35.3%	100%
Gen HR Mean (bpm)	—	90.5	—	68.4	99.7

Gen HR Std (bpm)	—	21.1	—	20.5	47.4
Dataset	MIT-BIH	MIT-BIH + PTB-XL	MIT-BIH+PTB-XL+Chapman	MIT-BIH+PTB-XL+Chapman	
Epochs	200	200	—	474 (early stop)	—

*Run 3 used same model as Run 2 — no code changes, just dataset fix attempt.

What We Fixed Before Run 4 (Phase 1 Improvements)

[OK] Things That Worked

1. FFD (Signal Quality): 42 → 21.7 — 48% improvement

The overall distribution quality of generated signals significantly improved. This is likely due to:

- Time-warp and amplitude augmentation helping the DiT learn more robust signal patterns
- Morphology gradient loss enforcing QRS slope sharpness
- 474 epochs with proper early stopping (no overfitting)

2. ReID Top-1 recovered: 5.9% → 11.8%

Patient identity preservation recovered from Run 2's drop. The frozen feature extractor (pre-trained contrastive encoder) successfully guides the DiT to maintain patient-specific morphology.

3. Early stopping worked perfectly

Model stopped at epoch 474 (max 500). Validation loss plateaued at 5.677 — the model was not overfitting. This confirms the training pipeline is now stable and self-regulating.

4. Contrastive pre-training: Excellent

Loss dropped from 0.040 → 0.0006 across 200 epochs on 75,540 segments across 3 datasets (MIT-BIH + PTB-XL + Chapman-Shaoxing). Identity encoder is now well-trained.

5. Chapman-Shaoxing dataset loaded

After fixing the download script (`.dat` file detection), Chapman's ~10K records were successfully included in training.

[!] Critical Bug Discovered: Spectral Loss Imbalance

This is the most important finding from Run 4.

W&B per-component loss breakdown revealed a catastrophic imbalance:

Loss Component	Final Value
noise_mse	0.099
signal_mse	0.180
identity	0.067
correlation	0.105
morphology	0.040
spectral	125.19 [!]

The spectral loss was 1,000× larger than all other components combined. This means:

- The optimizer spent ~99% of its gradient budget minimizing spectral loss
- All other losses (including identity and morphology) were effectively ignored
- The model learned to match FFT magnitudes but not heart rate or morphology
- This directly explains the HR MAE regression (9.3 → 31.4 bpm) — the model couldn't learn HR because identity/correlation losses were drowned out

Fix applied: Reduced spectral weight from 0.1 → 0.0001 (already committed to GitHub).

[~] HR Diversity Still Not Solved

Real ECG	Generated
HR 99.7 bpm Mean	68.4 bpm
HR 47.4 bpm Std	20.5 bpm

HR standard deviation is still only 43% of the real distribution. This means the model generates ECGs that are "too similar" in heart rate — it's not learning the full range from 50 to 150 bpm. Root causes:

1. DDIM determinism partially addressed ($\eta=0.75$) but HR conditioning is still implicit
2. Training data HR distribution may not be uniform (mostly 60-90 bpm range in MIT-BIH)
3. Without explicit HR conditioning, the model cannot be "told" to generate at specific heart rates

Run 4 Infrastructure Results

- **GPU:** NVIDIA A100 80GB PCIe — only used after clearing a competing 66GB process
- **Training time:** $\sim 856\text{s/epoch} \times 474 \text{ epochs} \approx 112 \text{ hours}$ (~ 4.7 days wall clock)
- **W&B:** Full per-component logging confirmed working — this was the key diagnostic tool
- **TensorBoard:** Also working in [checkpoints/runs/](#)
- **Datasets:** 75,540 total segments from MIT-BIH (8,592) + PTB-XL (56,300) + Chapman (10,648)

Plan for Run 5

Primary goal: Fix the 3 root causes of remaining failures.

Fix 1 — Spectral Loss Rescaling (Already Done)

`w_spectral`: 0.1 $\rightarrow$ 0.0001. Expected impact: HR MAE drops back below 10 bpm.

Fix 2 — Explicit Heart Rate Conditioning

Add HR as a first-class input to the DiT via AdaLN (same mechanism as timestep). During training, extract HR for each sample and condition the model on it. During inference, sample target HR from the real distribution. **This is the single most impactful remaining change.**

Fix 3 — RR-Interval Consistency Loss + HR Variance Regularization

Add two new losses that directly penalize HR diversity collapse:

- RR-interval loss: penalizes wrong mean HR
- HR variance loss: penalizes low batch-level HR std (< 47 bpm)

Research Prompts for Further Investigation

Use these prompts with Gemini/Claude to gather implementation details before Run 5:

Prompt 1 — Explicit HR Conditioning in DiT:

"I have a Diffusion Transformer (DiT) for ECG generation. Currently it takes patient identity embedding via AdaLN. I want to add explicit heart rate (HR in bpm) conditioning alongside the timestep. The model uses: timestep $\rightarrow$ sinusoidal embedding $\rightarrow$ AdaLN scale/shift for each transformer block. How do I add HR conditioning? Should I: (a) add HR embedding to the timestep embedding before AdaLN, (b) add a separate AdaLN layer just for HR, or (c) use cross-attention? What is the ECGTwin (2025) or DiffuSETS (2025) approach? Provide PyTorch pseudocode."

Prompt 2 — Differentiable RR-Interval Loss:

"I need a differentiable PyTorch loss function that estimates RR intervals from a raw 1D ECG signal (2500 samples at 500Hz, single lead, normalized). The loss should penalize mismatch between generated and real mean HR. I cannot use librosa or scipy (not differentiable). Options are: (a) soft R-peak detection via local max pooling, (b) autocorrelation-based HR estimation, (c) 1D CNN trained on peak detection. Which is most stable during backprop? Provide the full PyTorch implementation."

Prompt 3 — Loss Weight Balancing:

"I am training a multi-objective diffusion model for ECG generation with 6 loss components: noise_MSE (1.0), signal_MSE (1.0), identity_cosine (0.5), spectral_FFT (0.0001), correlation (0.2), morphology_gradient (0.3). W&B shows all are on similar scales now except spectral. What methods exist for automatic loss weight balancing in multi-task learning? Compare: GradNorm (Chen et al., 2018), PCGrad (Yu et al., 2020), CAGrad (Liu et al., 2021), MGDA (Desideri, 1988). Which is best for diffusion models? PyTorch example."